

- **Express**

- express 介绍

- Express 是一个第三方模块，用于快速搭建服务器（替代http模块）
- Express 是一个基于 Node.js 平台，快速、开放、极简的 web 开发框架。
- Express保留了http模块的基本API，使用express的时候，也能使用http的API
 - 使用express的时候，仍然可以使用http模块的方法，比如 `res.end()`、`req.url`
- express还额外封装了一些新方法，能让我们更方便的搭建服务器
- express提供了中间件功能，其他很多强大的第三方模块都是基于express开发的
- 文档地址：
 - [Express 官网](http://expressjs.com/)
 - [Express 中文文档（非官方）](http://www.expressjs.com.cn/)
 - [Express GitHub仓库](https://github.com/expressjs/express)
 - [菜鸟教程](https://www.runoob.com/w3cnote/express-4-x-api.html)
 - [腾讯云开发者手册](https://cloud.tencent.com/developer/doc/1079)
 - 百度自行搜索

- 安装 express

- 项目文件夹中，执行 ``npm i express``。即可下载安装express。

- 使用Express构造Web服务器

- 使用Express构建Web服务器步骤：
 - 加载 express 模块
 - 创建 express 服务器
 - 开启服务器
 - 监听浏览器请求并进行处理

```
1 // 使用express 搭建web服务器
2 // 1) 加载 express 模块
3 const express = require('express');
4
5 // 2) 创建 express 服务器
6 const app = express();
7
8 // 3) 开启服务器
9 app.listen(3006, () => console.log('express服务器开始工作了'));
10
11 // 4) 监听浏览器请求并进行处理
12
13 app.get('GET请求的地址', 处理函数);
14
```

```
15 app.post('POST请求的地址', 处理函数);
```

- express封装的新方法

- express之所以能够实现web服务器的搭建，是因为其内部对核心模块http进行了封装。
- 封装之后，express提供了非常方便好用的方法。
- 比如前面用到的 `app.get()` 和 `app.post()` 就是express封装的新方法。
- 下面再介绍一个 `res.send()` 方法
 - 该方法可以代替之前的 `res.end` 方法，而且比 `res.end` 方法更好用
 - `res.send()` 用于做出响应
 - 响应的内容同样不能为数字
 - 如果响应的是JS对象，那么方法内部会自动将对象转成JSON格式。
 - 而且会自动加Content-Type响应头
 - 如果已经做出响应了，就不要再再次做出响应了。

```
1 const express = require('express');
2 const app = express();
3 app.listen(3006, () => console.log('启动了'));
4
5 // 写接口
6 app.get('/api/test', (req, res) => {
7   // res.end('hello world, 哈哈'); // 响应中文会乱码，必须自己加响应头
8   // res.end(JSON.stringify({ status: 0, message: '注册成功' })); // 只能响应字符串或者buffer类型
9
10  // express提供的send方法，可以解决上面的两个问题
11  res.send({ status: 0, message: '注册成功' }); // send方法会自动设置响应头；并且会自动把对象转成JSON字符串
12 });
```

- 案例 - 登录注册接口

- 使用GIT管理项目

```
1 - bigevent-server
2   - index.js
3   - db.js
4   - .gitignore
5   - package.json
```

```
6      - package-lock.json
7      - node_modules      ---- 被忽略
```

- 搭建好项目目录之后。使用Git初始化。
- 设置忽略文件（`.gitignore`），这个忽略文件中记录的文件、文件夹不会添加到暂存区，不会提交到本地仓库，当然也就不会推送到远程仓库。

```
1 # git的忽略文件
2 # 忽略文件中指定的 文件、文件夹 不会被添加到暂存区，不会提交到本地仓库，不会推送到远程仓库
3 node_modules
```

- 设置好忽略文件之后，下一步add、commit、push。即可。
- 如果误把该忽略的文件add了，commit了。怎么办？
 - `git rm -r --cached 文件`（只移除本地仓库的文件，不删除工作区的文件）
 - `git add .`（重新添加一次）
 - `git commit -m 'xxx'`（重新提交即可）
 - 这样做完，如果发现vscode文件颜色没有变化，重启vscode再看看。
- 忽略文件的语法
 - [官方文档](<https://git-scm.com/book/zh/v2/Git-%E5%9F%BA%E7%A1%80-%E8%AE%B0%E5%B D%95%E6%AF%8F%E6%AC%A1%E6%9B%B4%E6%96%B0%E5%88%B0%E4%BB%93%E5% BA%93>)

```
1 # 只忽略根目录里面的 node_modules
2 /node_modules
3
4 # 忽略所有叫做 node_modules 的文件夹
5 node_modules/
6
7 # 忽略所有的 .a 文件
8 *.a
9
10 # 但跟踪所有的 lib.a，即便你在前面忽略了 .a 文件
11 !lib.a
12
13 # 只忽略当前目录下的 TODO 文件，而不忽略 subdir/TODO
14 /TODO
15
16 # 忽略任何目录下名为 build 的文件夹
```

```

17 build/
18
19 # 忽略 doc/notes.txt, 但不忽略 doc/server/arch.txt
20 doc/*.txt
21
22 # 忽略 doc/ 目录及其所有子目录下的 .pdf 文件
23 doc/**/*.pdf

```

◦ 创建数据表

字段	类型	长度	不是null	主键	其他
id	int		√	🔑	√自动递增
username	varchar	10	√		
password	char	32	√		
user_pic	longtext				
nickname	varchar	10			
email	varchar	30			

◦ 使用ApiPost模拟注册请求

注册接口 ▶ 接口说明 📄 🔄 同步 🔒 🗄️ 🚧 开发中

POST

http://localhost:3006/api/reguser

发送

保存

创建备份

Header

Query

Body

认证

预执行脚本

后执行脚本

Body参数

↑ 导入参数

↓ 导出参数

application/x-www-form-urlencoded

☒	username	Text	lisi	必填	类型	参数描述, 用于生成文档
☒	password	Text	123456	必填	类型	参数描述, 用于生成文档
	参数名	Text	参数值, 支持Mock字段变量	必填	类型	参数描述, 用于生成文档

◦ 写接口

```

1 // 完成接口项目
2 // 前面三行启动服务
3 const express = require('express');
4 const app = express();
5 app.listen(3006, () => console.log('启动了'));
6
7 // 配置 + 写接口
8 // ----- 注册接口 -----

```

```

9 // 请求体: username password
10 app.post('/api/reguser', (req, res) => {
11   // 1. 接口要接收数据
12   // 2. 判断账号是否已经被占用了
13   // 3. 如果没有被占用, 把账号密码添加到数据库
14 });

```

- 服务端使用 req.body 接收请求体
 - 请求体就是客户端提交的数据 (username和password) 。

```

1 // 完成接口项目
2 // 前面三行启动服务
3 const express = require('express');
4 const app = express();
5 app.listen(3006, () => console.log('启动了'));
6
7 // 配置 + 写接口
8
9 app.use(express.urlencoded({ extended: true }));
10
11 // ----- 注册接口 -----
12 // 请求体: username password
13 app.post('/api/reguser', (req, res) => {
14   // 1. 接口要接收数据
15   console.log(req.body); // { username: 'laotang', password: '123456' }
16   // 2. 判断账号是否已经被占用了
17   // 3. 如果没有被占用, 把账号密码添加到数据库
18 });

```

- 验证用户名是否存在
 - 思路: 根据用户名查询, 看是否能够查到数据。
 - 没有查询数据, 说明这个用户名不存在, 能够使用
 - 如果查到数据库, 说明这个用户名已经存在, 不能使用

```

1 // 完成接口项目
2 // 前面三行启动服务
3 const express = require('express');
4 const app = express();

```

```

5 app.listen(3006, () => console.log('启动了'));
6
7 // 配置 + 写接口
8
9 app.use(express.urlencoded({ extended: true }));
10
11 // ----- 注册接口 -----
12 // 请求体: username password
13 app.post('/api/reguser', (req, res) => {
14   // 1. 接口要接收数据
15   console.log(req.body); // { username: 'laotang', password: '123456' }
16   let { username, password } = req.body;
17   // 2. 判断账号是否已经被占用了
18   db('select * from user where username='${username}', (err, result) => {
19     if (err) throw err;
20     // console.log(result); // 查到信息, result是非空数组; 没有查到信息, result是空数组
21     if (result.length > 0) {
22       res.send({ status: 1, message: '用户名被占用了' });
23     } else {
24       // 没有被占用
25       // 3. 如果没有被占用, 把账号密码添加到数据库
26     }
27   })
28 });

```

◦ 完成注册

- 如果用户名可用, 则添加到数据表中, 完成注册

```

1 // ----- 注册接口 -----
2 // 请求体: username password
3 app.post('/api/reguser', (req, res) => {
4   // 1. 接口要接收数据
5   // console.log(req.body); // { username: 'laotang', password: '123456' }
6   let { username, password } = req.body;
7   // 2. 判断账号是否已经被占用了
8   db(`select * from user where username='${username}', (err, result) => {
9     if (err) throw err;
10    // console.log(result); // 查到信息, result是非空数组; 没有查到信息, result是空数组
11    if (result.length > 0) {
12      res.send({ status: 1, message: '用户名被占用了' });

```

```

13     } else {
14         // 没有被占用
15         // 3. 如果没有被占用, 把账号密码添加到数据库
16         db(`insert into user set username='${username}', password='${password}'`,
17         (e, r) => {
18             if (e) throw e;
19             res.send({ status: 0, message: '注册成功' });
20         });
21     });
22 });

```

◦ 对密码进行md5加密

- 安全起见, 数据表中不能存储明文密码。必须存储加密后的密码, 而且应该使用一种不可逆的加密方案。常用的加密方式是 md5。
- 使用方法
 - 下载安装第三方加密模块, 模块名 md5
 - `npm install md5`
 - 加载md5
 - `let md5 = require('md5')`
 - 对密码进行加密
 - `password = md5(password)`

◦ 客户端模拟登录请求

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3006/api/login
- Body Parameters:**

Checkbox	Parameter Name	Type	Value	Required	Type	Description
☒	username	Text	laotang	必填	类型	用户名
☒	password	Text	123456	必填	类型	密码
	参数名	Text	参数值, 支持Mock字段变量	必填	类型	参数描述, 用于生成文档
- Content-Type:** application/x-www-form-urlencoded

◦ 完成登录接口

- 接口要接收账号和密码 (前面已经配置好 `app.use(express.urlencoded({ extended: true })))`, 所以还是直接使用 `req.body`
- 对密码进行加密
- 使用账号和加密的密码当条件, 查询。

```

1 /**
2  * 登录接口
3  * 请求方式: POST

```

```

4  * 接口地址: /api/login
5  * 请求体: username | password
6  */
7  app.post('/api/login', (req, res) => {
8    // console.log(req.body); // { username: 'laotang', password: '123456' }
9    let { username, password } = req.body;
10   password = md5(password);
11   // 使用username和加密的密码当做条件, 查询。
12   let sql = `select * from user where username='${username}' and
password='${password}'`;
13   db(sql, (err, result) => {
14     if (err) throw err;
15     // console.log(result); // 没有查到结果得到空数组;  查到结果得到非空数组
16     if (result.length > 0) {
17       res.send({ status: 0, message: '登录成功' })
18     } else {
19       res.send({ status: 1, message: '账号或密码错误' })
20     }
21   })
22 });

```

◦ 创建token

- 使用第三方模块 jsonwebtoken 创建token字符串。
 - 下载安装 npm i jsonwebtoken
 - 加载模块 const jwt = require('jsonwebtoken');
 - 调用 jwt.sign() 方法创建token
 - 参数1: 必填, 对象形式; 希望在token中保存的数据
 - 参数2: 必填, 字符串形式; 加密的钥匙; 后续解密token的时候, 还需要使用。
 - 参数3: 可选, 对象形式; 配置项, 比如可以配置token的有效期
 - 参数4: 可选, 函数形式; 生成token之后的回调
 - 生成的token前面, 必须拼接 `Bearer ` 这个字符串。

```

1  if (result.length > 0) {
2    // 登录成功, 生成token
3    // 在token中保存用户的id
4    // token前必须加 "Bearer ", 注意空格
5    let token = 'Bearer ' + jwt.sign({ id: result[0].id }, 'sfsws23s', {
      expiresIn: '2h' });
6    res.send({ status: 0, message: '登录成功', token })
7  } else {

```



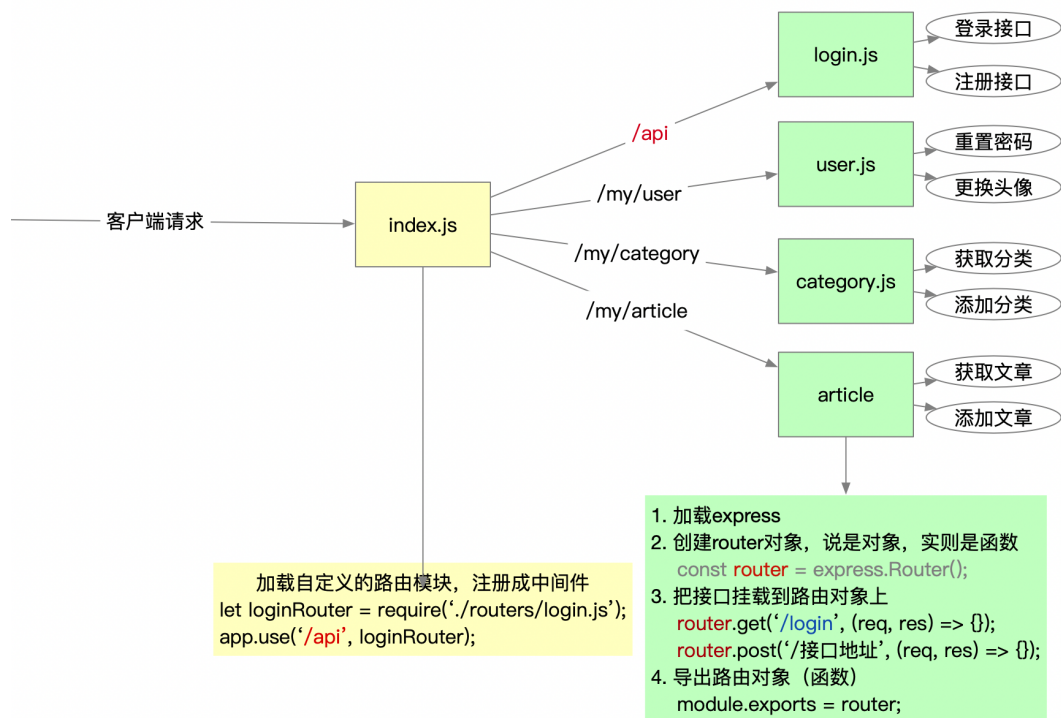
```
8   res.send({ status: 1, message: '账号或密码错误' })
9 }
```

- Express路由

- 路由：即请求和处理程序的映射关系。
- 使用路由的好处：
 - 降低匹配次数，提高性能
 - 分类管理接口，更易维护与升级
- 使用步骤：

```
1  /**
2   * 使用路由文件的步骤
3   * 1. 加载express模块
4   * 2. 创建 router 对象
5   * 3. 把接口挂载到 router 对象上
6   * 4. 导出 router 对象
7   *
8   * index.js 中
9   * 5. 加载路由模块，并注册成中间件
10  */
```

- 注意事项：
 - 路由文件如果没有导出 router，那么在 入口文件中不要注册中间件，否则报错
 - 哪个路由文件中使用了db，自己加载（谁用谁加载）



o login.js

```

1 // 加载所需的模块
2 const db = require('../db');
3 // const utils = require('utility'); // { noop: fun..., try: fun..., md5:
  fun..., sha1: fun... }
4 let { md5 } = require('utility');
5 let jwt = require('jsonwebtoken');
6
7 // ----- 使用路由的步骤 -----
8 // 1. 加载express
9 const express = require('express');
10 // 2. 创建路由对象，实则是个函数类型
11 const router = express.Router();
12
13 // 3. 写接口，把接口挂载到 router 上
14 router.post('/reguser', (req, res) => {});
15 router.post('/login', (req, res) => {});
16
17 // 一定要导出 router
18 module.exports = router;
  
```

o index.js

```

1 // 三行必须的代码，启动服务
2 const express = require('express');
3 const app = express();
4 app.listen(3006, () => console.log('启动了'));
5
6 // 下面一行的配置的意思，接收客户端提交的请求体；并且赋值给req.body (req.body = { 客户端提交的数据 })
7 app.use(express.urlencoded({ extended: true }));
8
9 // 加载自定义的路由模块，注册中间件
10 let loginRouter = require('./routers/login');
11 app.use('/api', loginRouter);

```

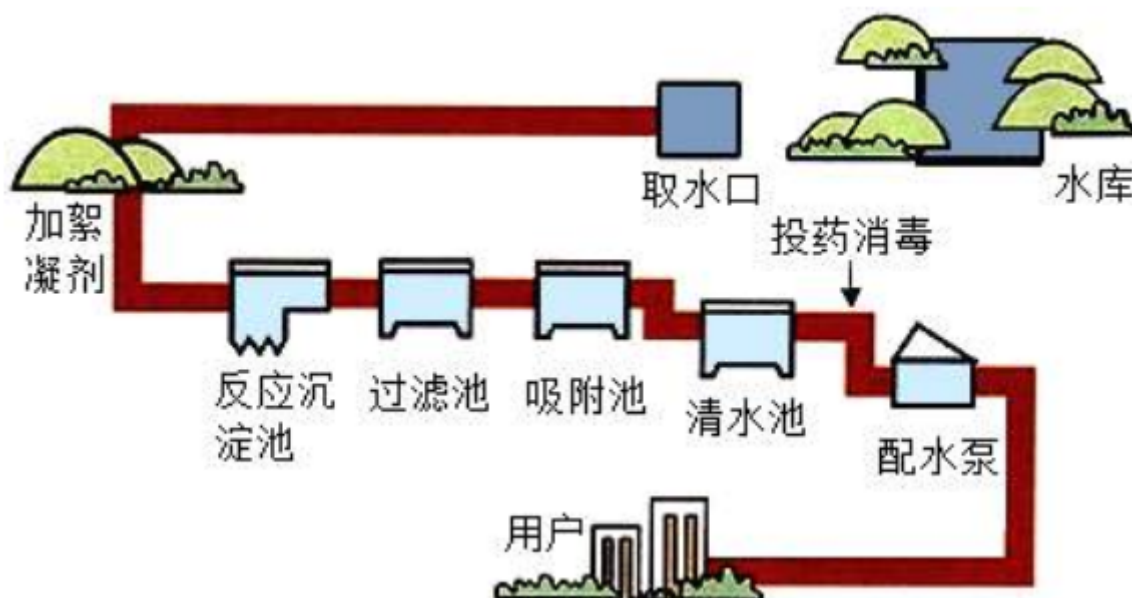
• Express中间件

◦ 中间件介绍

- 中间件(Middleware)，特指业务流程的中间处理环节。
- 中间件，是express最大的特色，也是最重要的一个设计
- 很多第三方模块，都可以当做express的中间件，配合express，开发更简单。
- 一个express应用，是由各种各样的中间件组合完成的
- 中间件，本质上就是一个函数

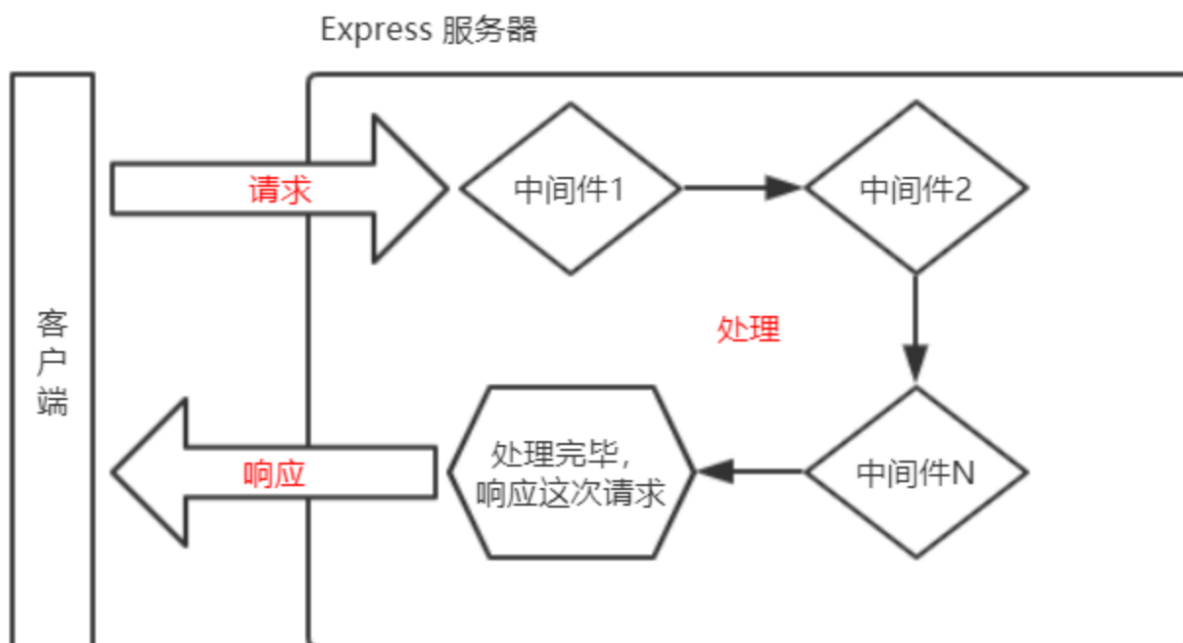
◦ 中间件原理

- 为了理解中间件，我们先来看一下我们现实生活中的自来水管的净水流程。



- 在上图中，自来水厂从获取水源到净化处理交给用户，中间经历了一系列的处理环节
- 我们称其中的每一个处理环节就是一个中间件。
- 这样做的目的既提高了生产效率也保证了可维护性。

- express中间件原理：



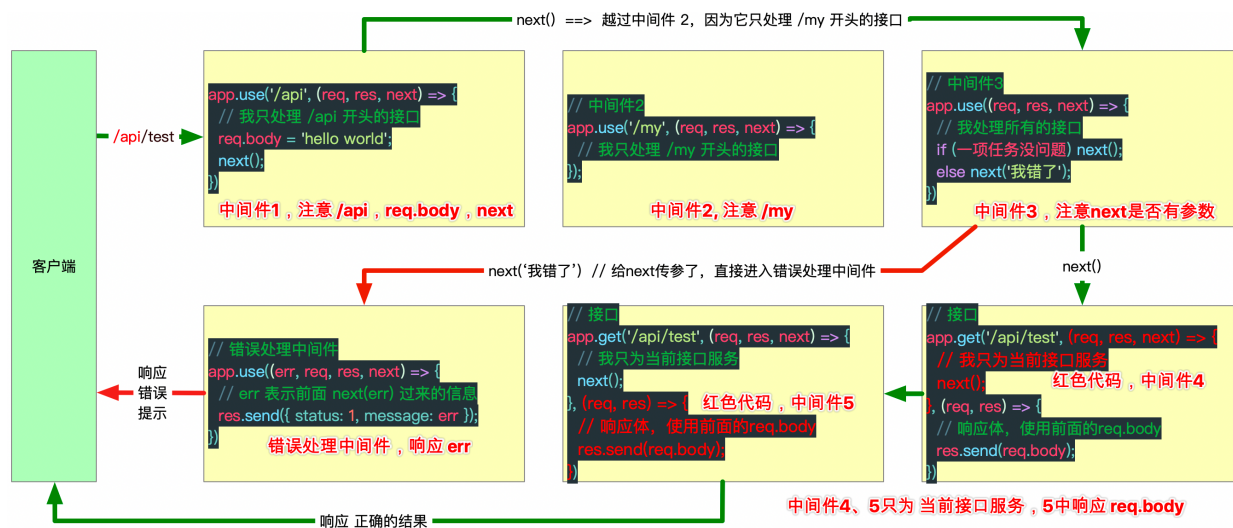
- 中间件的几种形式

```
1 // 下面的中间件, 只为当前接口 /my/userinfo 这个接口服务
2 app.get('/my/userinfo', 中间件函数);
3
4 // 下面的几个中间件, 是处理 /api/login 接口的
5 app.post('/api/login', 中间件函数, 中间件函数, 中间件函数, 中间件函数 .....);
6
7 // app.use 中的中间件, 可以处理所有的GET请求和所有的POST请求, 没有指定路径, 那么处理所有接口
8 app.use(中间件函数);
9
10 // 下面的中间件函数, 只处理 /api 开头的接口
11 app.use('/api', 中间件函数);
12
13 // 下面的中间件函数, 处理 /abcd 、 /abd 这两个接口
14 app.use('/abc?d', 中间件函数);
```

- 中间件语法

- 中间件就是一个函数
- 中间件函数中有四个基本参数, err、req、res、next
 - 如果写两个参数, 那么两个参数肯定是 req 和 res
 - 如果写三个参数, 那么三个参数肯定是 req, res 和 next
 - 如果写四个参数, 那么就是全部的参数, 这个中间件叫做错误处理中间件。
- 把写好的中间件函数, 传递给 `app.get()`、`app.post()`、`或app.use()`使用

- 中间件的特点



- 每个中间件函数，共享req对象、共享res对象，即所有的req对象是一个对象；所有的res是一个对象
 - 比如上述中间件1，为 req 对象添加了 body属性。中间件5中可以直接使用。
- 不调用`next()`，请求会卡在当前中间件；调用`next()`表示将请求交给下一个中间件处理。
 - 上述中间件中的`next()`保证了 请求 ~ 响应 能够完整的进行完。
 - 没有给`next()`传递参数，则正常进入下一个中间件。
 - 有给`next(err)`传递参数，则直接进入错误处理中间件
- 所有请求都可以进入使用`app.use()`注册的中间件，但要注意前缀
 - 中间件1，给出接口前缀`/api`，所以只有`/api`开头的接口才能进入
 - 中间件2，给出接口前缀`/my`，所以只有`/my`开头的接口才能进入
 - 中间件3，没有给出接口前缀，所以任何接口都能进入
- 错误处理中间件，必须传递 err、req、res、next四个参数，而且要放到所有接口的后面
 - 一般用于同一处理错误信息。
 - 错误处理中间件，也可以继续`next(错误)`，把错误交给后续的错误处理中间件处理。

中间件分类

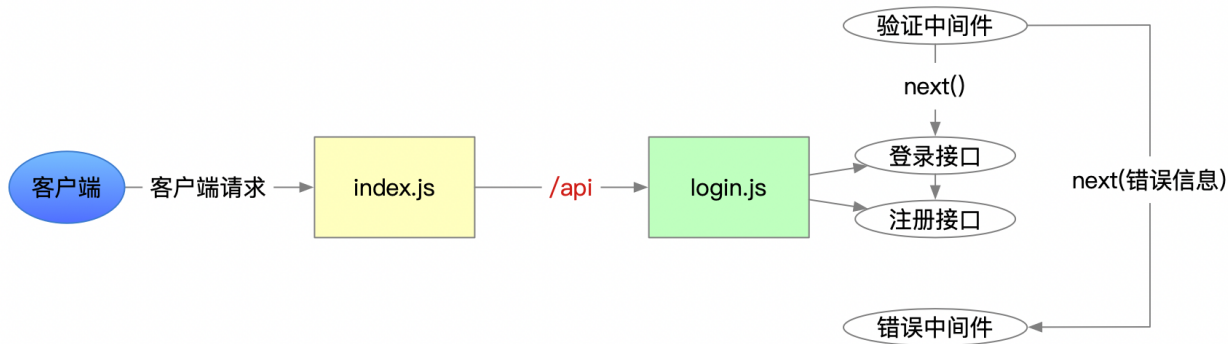
- 应用级别的中间件（index.js 中的中间件，全局有效）
- 路由级别的中间件（路由文件中的中间件，只在当前路由文件中有效）
- 错误处理中间件（四个参数都要填，一般放到最后）
- 内置中间件（express自带的，比如`express.urlencoded({ extended: true })`）
- 第三方中间件（比如multer、express-jwt、express-session、....）

案例 - 服务端数据验证

任务

- 客户端提交数据给服务器，服务器端也要进行验证。
- 开发领域，有一句话，叫做“永远不要相信客户端的数据”
- 验证，账号必须是2~10位，且只能使用数字、字母下划线组合，且必须是字母开头
- 验证，密码必须是6~12位，且不能有空格

验证流程



◦ 验证中间件

```

1 // 必须在这里，注册中间件，完成数据的验证
2 router.use((req, res, next) => {
3   // 获取username和password
4   let { username, password } = req.body;
5   // 验证用户名
6   if (!/^[a-zA-Z][0-9a-zA-Z_]{1,9}$/.test(username)) {
7     next('用户名只能包含字母下划线，长度2~10位，字母开头');
8   } else if (!/^\S{6,12}$/.test(password)) {
9     next('密码6~12位且不能出现空格');
10  } else {
11    next();
12  }
13 });

```

◦ 错误处理中间件

```

1 // 错误处理中间件
2 router.use((err, req, res, next) => {
3   // err 就是前面next过来的参数值
4   res.send({ status: 1, message: err });
5 });

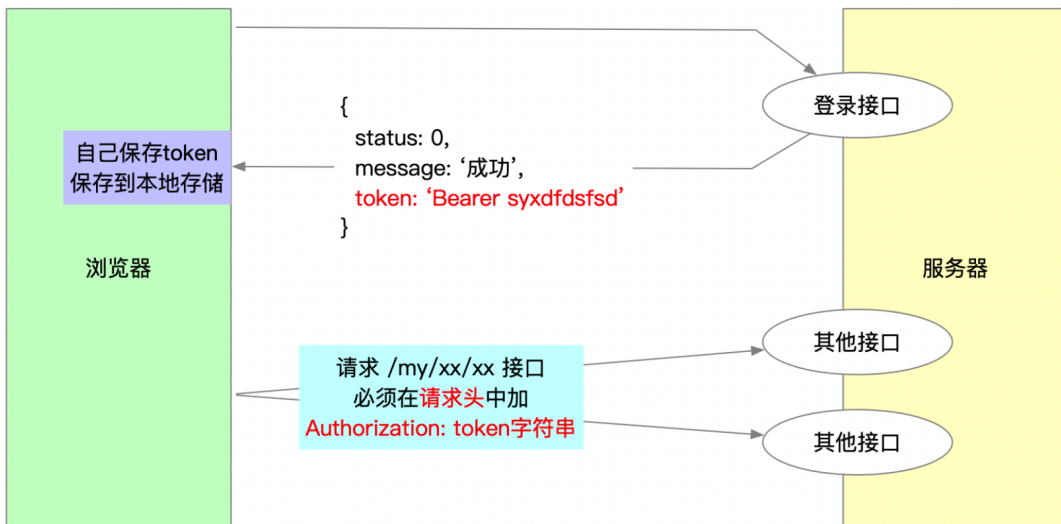
```

• 服务端身份认证

◦ JWT原理回顾

- 对于前后端分离模式的开发，大多使用 JWT (json web token) 进行身份认证。
- 前面已经讲过JWT的原理了，使用下图回顾一下。

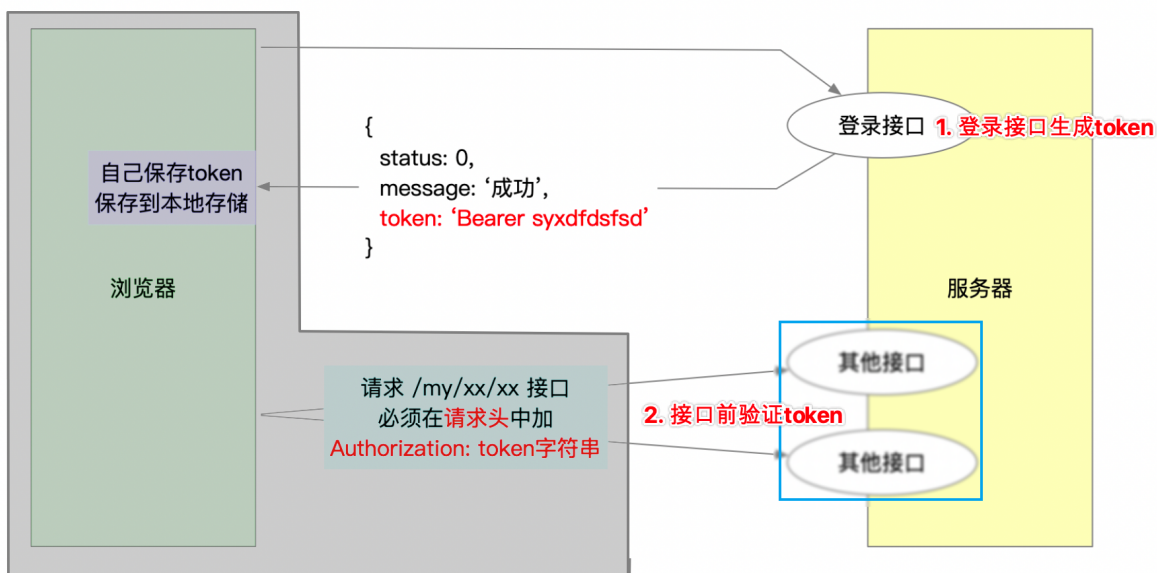
jsonwebtoken 身份认证机制，简称 JWT 身份认证机制
尤其适合前后端分离模式的开发



○ 分析服务端该做什么

- 大事件中已经把前端该做的完成了，剩下的就是后端的任务了。
- 前端：
 - 登录成功后，保存token
 - 请求 `/my/xxx` 接口时，在请求头中加入 `Authorization` 字段，值为token。
- 后端：
 - 登录接口生成token
 - 判断，如果客户端请求的是 `/my/xxx` 接口，要解析并验证token的真伪

jsonwebtoken 身份认证机制，简称 JWT 身份认证机制
尤其适合前后端分离模式的开发



○ 代码实现认证

- 选择使用 [express-jwt](https://gitee.com/mirrors_auth0/express-jwt) 第三方模块进行身份认证。从模块名可以看出，该模块是专门配合express使用的。
- 下载安装：`npm i express-jwt`
- 后端 index.js 中，当接收到一个请求后，先解析并验证token。

```

1  const jwt = require('express-jwt');
2  // app.use(jwt().unless());
3  // jwt() 用于解析token, 并将 token 中保存的数据 赋值给 req.user
4  // unless() 完成身份认证
5  app.use(jwt({
6    secret: 'sfsws23s', // 生成token时的 钥匙, 必须统一
7    algorithms: ['HS256'] // 必填, 加密算法, 无需了解
8  }).unless({
9    path: ['/api/login', '/api/reguser'] // 除了这两个接口, 其他都需要认证
10 }));

```

- 上述代码完成后, 当一个请求发送到服务器后, 就会验证请求头中的 Authorization 字段了。
 - 如果没有问题
 - 将token中保存的 用户id 赋值给 req.user
 - next()。
 - 如果有问题
 - next(错误信息)。
- 所以, 还需要在index.js 最后, 加入错误处理中间件, 来提示token方面的错误。文档中抄下来, 修改。

```

1  app.use((err, req, res, next) => {
2    if (err.name === 'UnauthorizedError') {
3      // res.status(401).send('invalid token...');
4      res.status(401).send({ status: 1, message: '身份认证失败! ' });
5    }
6  });

```

• 案例 - 个人中心接口

- 设计数据表
 - 因为已经做过登录注册了, 所以, 这里不用再次创建了。
- 使用路由模块
 - /routers/user.js

```

1  // user路由文件
2  const express = require('express');
3  const router = express.Router();

```



```

4
5 // 导出
6 module.exports = router;

```

- index.js中加载路由模块，注册中间件

- index.js

```

1 app.use('/my/user', require('./routers/user'));

```

- 获取用户信息接口

- ApiPost模拟请求

获取用户信息接口

GET http://localhost:3000/my/user/userinfo 发送 保存

Header Query Body 认证 预执行脚本 后执行脚本

Header参数 导入参数 导出参数

☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJp	必填 类型 参数描述, 用于生成文
	参数名	参数值, 支持Mock字段变量	必填 类型 参数描述, 用于生成文

先登录，然后复制token

- 接口代码：

```

1 // ***** 获取用户信息 ***** /
2 /**
3  * 请求方式: GET
4  * 接口地址: /my/user/userinfo
5  * 参数: 无
6  */
7 router.get('/userinfo', (req, res) => {
8   // console.log(req.user); // { id: 1, iat: 1611537302, exp: 1611544502 }
9   // return;
10  // 查询当前登录账号的信息，并不是查询所有人的信息
11  db('select * from user where id=' + req.user.id, (err, result) => {
12    if (err) throw err;
13    res.send({
14      status: 0,
15      message: '获取用户信息成功',
16      data: result[0]
17    })
18  });

```

19 });

■ 更新用户信息接口（有问题）

- 客户端模拟请求：

更新用户信息接口

POST http://localhost:3006/my/user/userinfo

Header Query Body 认证 预执行脚本 后执行脚本

Body参数 导入参数 导出参数

application/x-www-form-urlencoded

☒	nickname	Text	老汤	必填	类型	参数描述，用于生成文档
☒	email	Text	23323@qq.com	必填	类型	参数描述，用于生成文档
☒	id	Text	1	必填	类型	必须是登录账号的id
	参数名	Text	参数值，支持Mock字段变量	必填	类型	参数描述，用于生成文档

- 代码实现：

```
1 // ***** 更新用户信息 ***** /
2 /**
3  * 请求方式: POST
4  * 接口地址: /my/user/userinfo
5  * Content-Type: application/x-www-form-urlencoded
6  * 请求体: email | nickname | id
7  */
8 router.post('/userinfo', (req, res) => {
9   // console.log(req.body); // { nickname: '老汤', email: '23323@qq.com', id: '1' }
10  let { id, nickname, email } = req.body;
11  if (id !== req.user.id) return res.send({ status: 1, message: '无权更新' });
12  let sql = `update user set nickname='${nickname}', email='${email}' where id=${id}`;
13  db(sql, (err, result) => {
14    if (err) throw err;
15    res.send({ status: 0, message: '更新用户信息成功' });
16  })
17 });
```

■ 更换头像接口

- 客户端模拟请求：



- 代码实现：

```
1 // ***** 更换头像接口 ***** /
2 /**
3  * 请求方式: POST
4  * 接口地址: /my/user/avatar
5  * Content-Type: application/x-www-form-urlencoded
6  * 请求体: avatar
7  */
8 router.post('/avatar', (req, res) => {
9   // console.log(req.body); // { avatar:
   'data:image/png;base64,iVBORw0KGgoAAAANSU'
10   let sql = `update user set user_pic='${req.body.avatar}' where
   id=${req.user.id}`;
11   db(sql, (err, result) => {
12     if (err) throw err;
13     res.send({ status: 0, message: '更换头像成功' })
14   })
15 });
```

■ 重置密码接口

- 客户端模拟请求：



- 代码实现：

```
1 //更新密码
```

```

2 router.post('/updatepwd', function (req, res) {
3     let {
4         oldPwd,
5         newPwd
6     } = req.body
7     oldPwd = md5(oldPwd)
8     newPwd = md5(newPwd)
9     db(`UPDATE user SET password='${newPwd}' WHERE id='${req.user.id}' and
password='${oldPwd}'`, function (error, result) {
10         if (error) throw error
11         if (result.affectedRows > 0) {
12             res.send({
13                 status: 0,
14                 message: "更新密码成功"
15             })
16         } else {
17             res.send({
18                 status: 1,
19                 message: "旧密码有误, 更新密码失败"
20             })
21         }
22     })
23 })

```

• ApiPost认证

- 登录之后，把token字符串，保存到ApiPost的全局变量中。

登录接口

POST http://localhost:3006/api/login

当登录成功后，执行的脚步

Header Query Body 认证 预执行脚本 后执行脚本

```

1 // 登录完成之后，把token字符串保存起来。
2
3 // apt.globals.set("token", response.json.token.substr(7));
4
5 apt.globals.set("token", response.json.token.replace('Bearer ', ''));

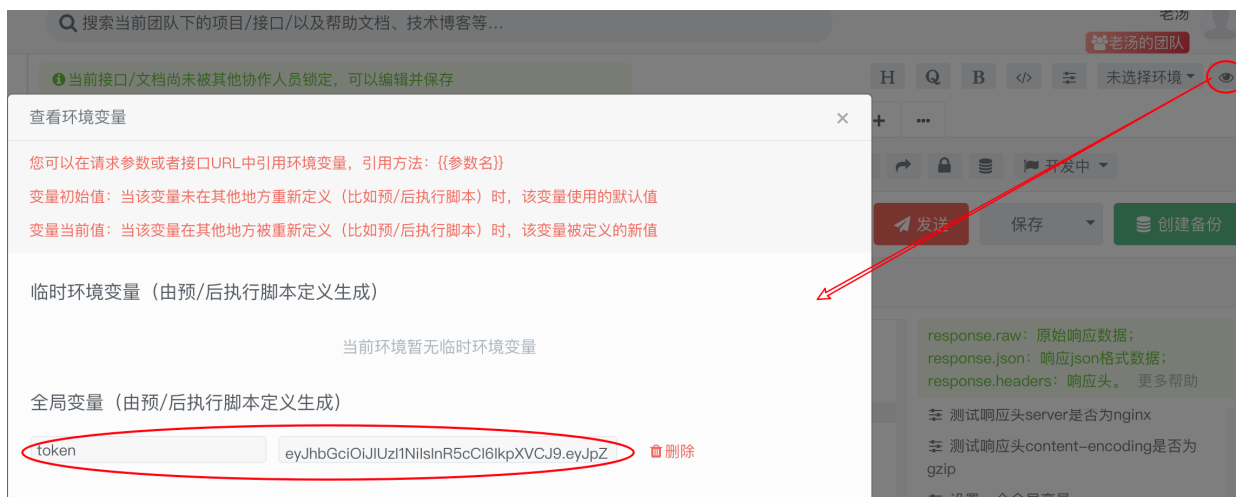
```

在全局变量中，保存一个token，值是token字符串，但是截取掉了Bearer空格

response.raw: 原始响应数据;
response.json: 响应json格式数据;
response.headers: 响应头。 更多帮助

测试响应头server是否为nginx
测试响应头content-encoding是否为gzip
设置一个全局变量
获取一个全局变量
删除一个全局变量

- 登录之后，就会在ApiPost的全局变量中，保存上一个token。可以点击小眼睛查看：



- 其他需要认证的接口，选择认证，Bearer auth认证，填写 {{token}} 即可。
- ApiPost发送请求的时候，会自动携带Authorization 这个请求头，并且会在token字符串前加上“Bearer ”，从而完成身份认证。



• 案例 - 类别管理接口

- 设计数据表并添加模拟数据
 - 导入SQL。（user、category、article表中的数据有关联。必须全部导入，然后使用账号 admin、密码 admin登录）
- 使用路由模块
 - /routers/category.js

```

1 // category路由文件
2 const express = require('express');
3 const router = express.Router();
4
5
6 // 导出
7 module.exports = router;

```

- index.js中加载路由模块，注册中间件
 - index.js

```
1 app.use('/my/category', require('./routers/category'));
```

- 完成获取分类列表数据的接口
 - 注意加载db.js

```
1 // ----- 获取分类的接口 -----
2 /**
3  * 请求方式:GET
4  * 接口地址: /my/category/list
5  * 参数: 无
6  */
7 router.get('/list', (req, res) => {
8     // 调用db函数, 查询所有的分类
9     db('select * from category', (err, result) => {
10         if (err) throw err;
11         // 没有错误的话, 做出响应
12         res.send({
13             status: 0,
14             message: '获取分类成功',
15             data: result
16         });
17     });
18 });
```

- 删除分类的接口
 - 客户端发送请求, 并且传递id参数



- 服务端代码:

```
1 // ----- 删除分类的接口 -----
2 /**
```

```

3  * 请求方式:GET
4  * 接口地址: /my/category/delete
5  * 请求参数: id(分类id), 类型querystring
6  */
7  router.get('/delete', (req, res) => {
8      // 获取id参数
9      var id = req.query.id;
10     // 删除数据表中的数据
11     db('delete from category where id=' + id, (err, result) => {
12         // 做出响应
13         if (err) throw err;
14         if (result.affectedRows > 0) {
15             res.send({status: 0, message: '删除分类成功'});
16         } else {
17             res.send({status: 1, message: '删除分类失败'})
18         }
19     });
20 });

```

◦ 添加分类的接口

添加分类接口

▶ 接口说明 □ 同步 ↺ 开发中 ▼

POST http://localhost:8888/my/category/add 发送 保存 创建备份

Header Query **Body** 认证 预执行脚本 后执行脚本

▼ Body参数 ↑ 导入参数 ↓ 导出参数

名称	类型	必填	描述
name	Text	是	类别名称
alias	Text	是	类别别名

Content-Type: application/x-www-form-urlencoded

■ 服务端代码:

- 由于 index.js 中, 有 `app.use(express.urlencoded({ extended: true }))`, 所以这里直接使用 `req.body` 接收请求体。

```

1  // ----- 添加分类的接口 -----
2  /**
3   * 请求方式:POST
4   * 接口地址: /my/category/add
5   * 请求参数: name(类别名称) | alias(类别别名)
6   * Content-Type: application/x-www-form-urlencoded
7   */
8  router.post('/add', (req, res) => {
9      // 1. 接收客户端提交的数据 (name和alias)

```

```

10 // console.log(req.body); // { name: '娱乐', alias: 'yule' }
11 let { name, alias } = req.body;
12 // 2. 添加到数据库
13 let sql = `insert into category set name='${name}', alias='${alias}'`;
14 db(sql, (err, result) => {
15   if (err) throw err;
16   // 3. 做出响应
17   res.send({ status: 0, message: '添加分类成功' })
18 });
19 });

```

◦ 更新分类接口

POST

http://localhost:8888/my/category/update

发送

保存

创建备份

Header

Query

Body

认证

预执行脚本

后执行脚本

Body参数

导入参数

导出参数

application/x-www-form-urlencoded

☒	id	Text	1	必填	类型	参数描述, 用于生
☒	name	Text	科技	必填	类型	参数描述, 用于生
☒	alias	Text	keji	必填	类型	参数描述, 用于生
	参数名	Text	参数值, 支持Mock字段变量	必填	类型	参数描述, 用于生

■ 服务端代码:

```

1 // ----- 修改分类的接口 -----
2 /**
3  * 请求方式:POST
4  * 接口地址: /my/category/update
5  * 请求参数: name(类别名称) | alias(类别别名) | id(分类的id)
6  * Content-Type: application/x-www-form-urlencoded
7  */
8 router.post('/update', (req, res) => {
9   // 1. 接收客户端提交的数据 (id、name、alias)
10  // console.log(req.body); // { id: '1', name: '科技', alias: 'keji' }
11  let { id, name, alias } = req.body;
12  // 2. 执行update语句, 修改数据
13  let sql = `update category set name='${name}', alias='${alias}' where id=${id}`;
14  db(sql, (err, result) => {
15    if (err) throw err;
16    if (result.affectedRows > 0) {
17      res.send({ status: 0, message: '修改分类成功' })

```



```

18     } else {
19         res.send({ status: 1, message: '修改分类失败' })
20     }
21 })
22 });

```

- 案例 - 文章相关接口

- 使用路由模块
 - /routers/article.js

```

1 // category路由文件
2 const express = require('express');
3 const router = express.Router();
4
5
6 // 导出
7 module.exports = router;

```

- index.js中加载路由模块，注册中间件
 - index.js

```

1 app.use('/my/article', require('./routers/article'));

```

- 分页获取文章接口
 - 注意引入 db.js

```

1 // ----- 分页获取文章列表 -----
2 // 接口要求:
3 /**
4  * 请求方式: GET
5  * 请求的url: /my/article/list
6  * 请求参数:
7  *   - pagenum -- 页码值
8  *   - pagesize -- 每页显示多少条数据
9  *   - cate_id -- 文章分类的Id
10  *   - state -- 文章的状态, 可选“草稿”或“已发布”

```

```

11  */
12  router.get('/list', (req, res) => {
13      // console.log(req.query);
14      // 设置变量, 接收请求参数
15      let { pagenum, pagesize, cate_id, state } = req.query;
16      // console.log(cate_id, state);
17      // 根据cate_id 和 state制作SQL语句的条件
18      let w = '';
19      if (cate_id) {
20          w += ` and cate_id=${cate_id} `;
21      }
22      if (state) {
23          w += ` and state='${state}' `;
24      }
25      // 分页查询数据的SQL (该SQL用到了连表查询, 并且使用了很多变量组合)
26      let sql1 = `select a.id, a.title, a.state, a.pub_date, c.name cate_name from
article a
27      join category c on a.cate_id=c.id
28      where author_id=${req.user.id} and is_delete=0 ${w}
29      limit ${pagenum - 1} * pagesize, ${pagesize}`;
30      // 查询总记录数的SQL, 查询条件和前面查询数据的条件 必须要一致
31      let sql2 = `select count(*) total from article a
32      join category c on a.cate_id=c.id
33      where author_id=${req.user.id} and is_delete=0 ${w}`;
34      // 分别执行两条SQL (因为db查询数据库是异步方法, 必须嵌套查询)
35      db(sql1, (err, result1) => {
36          if (err) throw err;
37          db(sql2, (e, result2) => {
38              if (e) throw e;
39              res.send({
40                  status: 0,
41                  message: '获取文章列表数据成功',
42                  data: result1,
43                  total: result2[0].total
44              });
45          })
46      })
47  });

```

- 添加文章接口

```

1 // ----- 添加文章接口 -----
2 /**
3  * 接口地址: /my/article/add
4  * 请求方式: POST
5  * 请求体: title | content | cate_id | state | cover_img
6  * Content-Type: multipart/form-data
7  */

```

- 这是我们遇到的第一个请求体为FormData类型的接口。
- 服务端获取FormData类型的数据，需要使用第三方模块 [multer] (<https://github.com/expressjs/multer/blob/master/doc/README-zh-cn.md>)。
- 安装: `npm i multer`
- 加载: `const multer = require('multer')`
- 配置上传文件路径: `const upload = multer({ desc: 'uploads/' })`
- 接口中使用:

```

1 router.post('/add', upload.single('cover_img'), (req, res) => {
2   // upload.single() 方法用于处理单文件上传
3   // cover_img 图片字段的名字
4
5   // 通过 req.body 接收文本类型的请求体, 比如 title,content等
6   // 通过 req.file 获取上传文件信息
7 });

```

- 只要客户端请求这个接口，就会自动创建 `uploads` 文件夹，并把文件上传到该文件夹。
- 此时，可以使用ApiPost测试：



- 注意，大事件接口文档规定，客户端只能提交 "title、content、cate_id、state、cover_img" 5个值。
- 而，article 数据表中，还要求添加 `author\_id` (用户id)、`pub\_date` (发布时间)，这两个字段的值只能自己来添加了。

- 发布时间的处理，需要使用 `moment` 模块，自行安装。

```
1 var multer = require('multer')
2 var upload = multer({ dest: 'uploads/' }); // 配置上传文件的目录
3 const moment = require('moment');
4
5 router.post('/add', upload.single('cover_img'), (req, res) => {
6   // req.body 表示文本信息
7   // req.file 表示上传的文件信息
8   // console.log(req.file); // req.file.filename 表示上传之后的文件名
9
10  // 把数据添加到数据表中存起来
11  // req.body = { title: 'xx', content: 'xx', cate_id: 1, state: 'xx' }
12  let { title, content, cate_id, state } = req.body;
13  // 其他字段
14  let pub_date = moment().format('YYYY-MM-DD HH:mm:ss');
15  let cover_img = req.file.filename;
16  let author_id = req.user.id;
17  // console.log(obj);
18  // return;
19  let sql = `insert into article set title='${title}', content='${content}',
cate_id=${cate_id}, state='${state}', pub_date='${pub_date}',
cover_img='${cover_img}', author_id=${author_id}`;
20  db(sql (err, result) => {
21    if (err) throw err;
22    if (result.affectedRows > 0) {
23      res.send({ status: 0, message: '发布成功' })
24    } else {
25      res.send({ status: 1, message: '发布失败' })
26    }
27  })
28 });
```

○ 删除文章接口

- 客户端模拟请求：

删除文件接口

GET

http://localhost:3006/my/article/delete/3

发送

保存

创建

Header

Query

Body

认证

预执行脚本

后执行脚本

Query参数

导入参数

导出参数

url参数，直接写到url后面即可

这也是GET请求，参数的一种形式

参数名	参数值，支持Mock字段变量	必填	类型	参数描述，用于生成文档

- 删除，可以做成“软删除”效果。即不是真的删除，而是把 is_delete 字段改为 1。
 - is_delete = 0，正常的文章。获取文章的时候，只获取 id_delete=0 的文章。
 - is_delete = 1, 表示已删除的文章。

```
1 // ----- 删除文件接口 -----
2 /**
3  * 请求方式: GET
4  * 接口地址: /my/article/delete/2
5  * 请求参数: id, url参数
6  */
7 // router.get('/delete/:id/:age/:name', (req, res) => {
8 router.get('/delete/:id', (req, res) => {
9   let id = req.params.id;
10   let sql = `update article set is_delete=1 where id=${id} and
    author_id=${req.user.id}`;
11   db(, (err, result) => {
12     if (err) throw err;
13     if (result.affectedRows > 0) {
14       res.send({ status: 0, message: '删除成功' })
15     } else {
16       res.send({ status: 1, message: '删除失败' })
17     }
18   })
19 });
```

更新文章接口

```
1 router.post('/update', upload.single('cover_img'), (req, res) => {
2   // 和添加文章接口差不多，要注意，客户端多提交了文章id，这是我们修改文章的条件
3   // req.body = { title: 'xx', content: 'xx', cate_id: 1, state: 'xx', id: 6 }
4   let { title, content, cate_id, state, id } = req.body;
5   // 其他字段（发布时间，不是修改时间，所以不需要改了，用户id也不需要改）
6   let cover_img = req.file.filename;
```

```

7 // console.log(obj);
8 // return;
9 let sql = `update article set title='${title}', content='${content}',
cate_id=${cate_id}, state='${state}', cover_img='${cover_img}' where id=${id}`;
10 db(sql, (err, result) => {
11   if (err) throw err;
12   if (result.affectedRows > 0) {
13     res.send({ status: 0, message: '修改文章成功' })
14   } else {
15     res.send({ status: 1, message: '修改文章失败' })
16   }
17 })
18 });

```

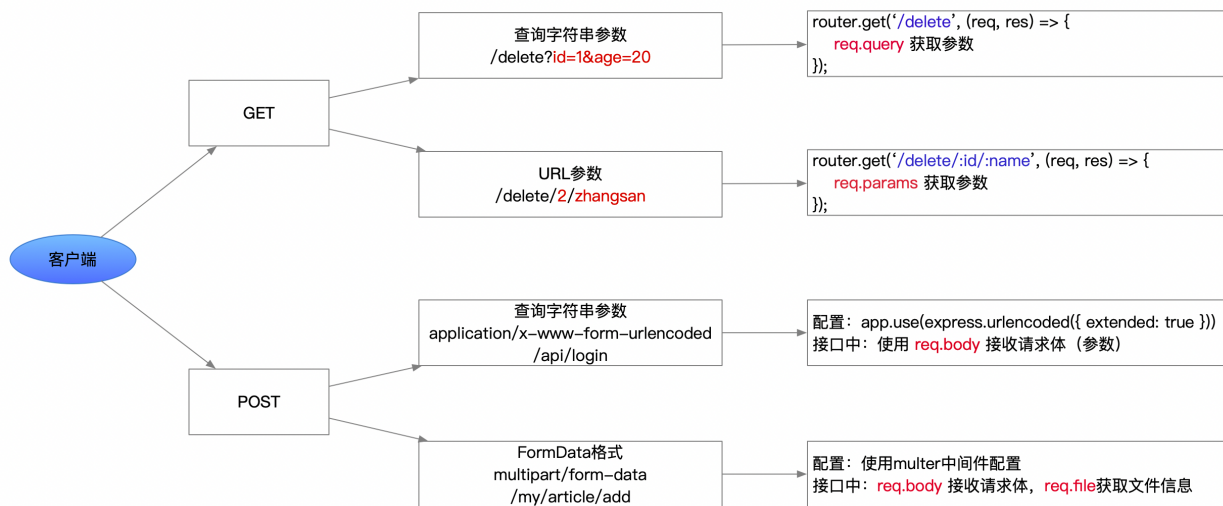
- 根据id获取一篇文章接口

```

1 // ----- 根据id获取一篇文章接口 -----
2 router.get('/:id', (req, res) => {
3   // 怎么获取url中的参数, 答, 使用express提供的 req.params
4   // console.log(req.params.id);
5   db('select * from article where id='+req.params.id, (err, result) => {
6     if (err) throw err;
7     res.send({
8       status: 0,
9       message: '获取文章成功',
10      data: result[0]
11    })
12  })
13 });

```

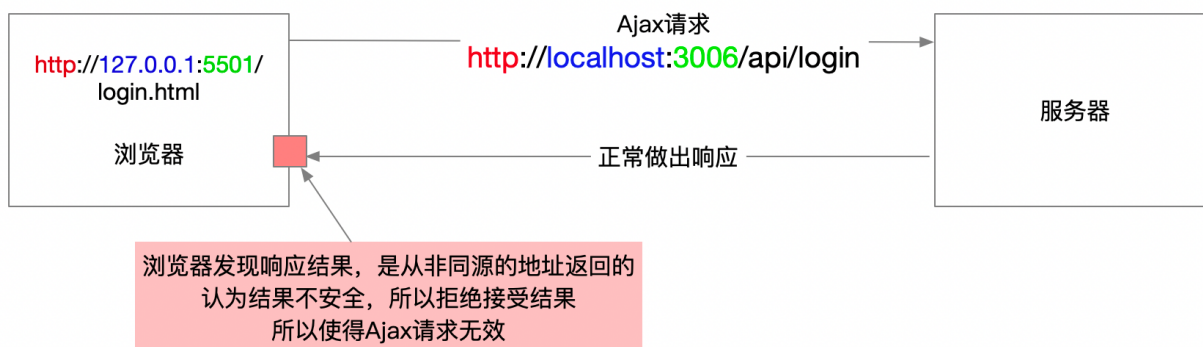
- 小结Express接收客户端数据



• 使用前端代码测试接口

◦ 同源策略

- 同宗同源：比如，你和你的亲兄弟、亲姐妹，叫叫做同源。
- 浏览器中，也有同源策略。指的是打开页面的URL和Ajax请求的URL比较，比较他俩是否同源。
- 比如，打开页面的URL：<http://127.0.0.1:5501/login.html>
- 发送Ajax请求的URL：<http://localhost:3006/api/login>
- 判断两个URL是否同源，查看协议、主机地址、端口号，如果这三项都相同，则称这两个URL同源，否则非同源。
- 如果非同源，则以下三种行为受到限制：
 - DOM无法操作
 - cookie不会自动携带
 - Ajax请求无效



- 这就属性跨域请求，即违反了同源策略的请求，就是跨域请求。

◦ 解决跨域-CORS

- CORS，叫做跨域资源共享，是XHR2.0中提出的一种新的解决跨域的方案。从IE10开始支持。
- CORS方案的实现，是通过服务器的响应头来实现的。
- 服务器要设置：`Access-Control-Allow-Origin: '\*或者一个具体的源'`
- 这里直接使用第三方模块 cors 来解决跨域。
 - 安装 `npm i cors`
 - 加载 `const cors = require('cors');`
 - 使用 `app.use(cors());` // 注意，必须把这个中间件，放到最前面。